

Statistical Programming Languages

Exercises: Day 5

Programming

Exercise 1 (A simple function):

Consider the function $f: \mathbb{R} \rightarrow \mathbb{R}$, $f(x) = \sin(x^2)$.

- a) Implement the function f in R.
- b) Evaluate $f(x)$ at points $x = 0$, $x = 3$, $x = -2$.
- c) Plot f on the interval $[-10, 10]$.

Exercise 2 (Fibonacci numbers):

Write a recursive function `fibonacci()`, which takes one argument $n \in \mathbb{N}$ and returns the n -th Fibonacci number y_n with $y_1 := 1$, $y_2 := 2$ and $y_m := y_{m-1} + y_{m-2} \ \forall m > 2$. (Advanced: Do not use loops.)

Exercise 3 (Timing):

- a) Write a function `my_sort()` that takes a numeric vector as input and returns the same values in increasing order without using the functions `sort()` or `order()`. (Hint: Use the `min()` function.)
- b) The function `sort()` does the same as your function `my_sort()`. Compare the run time of both functions for a large input vector using `system.time()`.

Exercise 4 (The `apply()` family):

Compute the minimum and the maximum of the variable `Sepal.Length` for each species of the `iris` data using an appropriate function from the `apply()` function family.

Exercise 5 (Simulating a stochastic process):

- a) Generate 100 random samples $x = (x_1, \dots, x_{100})$ from a normal distribution with $\mu = 0$ and $\sigma = 2$. Compute $y = (y_1, \dots, y_{100})$, such that $y_1 = 0$ and for $i = 1, \dots, 99$:

$$y_{i+1} = \begin{cases} y_i + x_i, & x_i \geq 0, \\ y_i - 2x_i, & -1 < x_i < 0, \\ y_i - x_i, & x_i \leq -1. \end{cases}$$

- b) Plot both stochastic processes x and y .

Exercise 6 (Bootstrap):

The confidence interval for the correlation coefficient between two variables can be estimated based on bootstrap:

- (i) Generate 1000 bootstrap samples* of length n , where n is the number of observations of the variables.
- (ii) Compute the (pearson) correlation coefficient for each bootstrap sample.
- (iii) Compute the 95% confidence interval of these correlation coefficients by means of the function `quantile(..., c(0.025, 0.975))`.

Estimate the confidence interval for the correlation coefficient between the variables `eruptions` and `waiting` from the built-in data set `faithful`

- a) using a `for` loop,
- b) based on functions of the `apply` family. (Hint: `?replicate`)

Compare the running times of both implementations using the function `system.time()`.

*A bootstrap sample is defined as a sample with replacement of the original sample of individuals. (Hint: `?sample`)

Exercise 7 (Tic-tac-toe):

The game *Tic-tac-toe* is played by two persons, who take turns marking the cells in a 3×3 grid. The player who succeeds in placing three of their marks in a horizontal, vertical, or diagonal line is the winner. The game ends in a tie, if there is no free cell left and none of the players has won.

Write a function `tic_tac_toe` that enables 2 persons to play Tic-tac-toe via the R console. It should fulfill the following:

- The function has no arguments. The basic structure is:

```
tic_tac_toe <- function() {  
  # Your part  
}
```

- When the function gets called, the following message is printed in the console (requesting Player 1 to make his move) and an empty playing field is plotted (in the Plots-tab of RStudio). (Hint: `?cat`)

```
> tic_tac_toe()  
In each move you have to choose one cell 1-9.  
Player 1:
```

The empty playing field should look like this:

7	8	9
4	5	6
1	2	3

- A player executes a move by inserting the number of the cell which should be marked when requested by the console. (Hint: `?scan`)
- The (partial) output in the console during the playing process should be as follows (inputs can differ of course):

```
> tic_tac_toe()  
In each move you have to choose one cell 1-9.  
Player 1:  
1: 5  
Player 2:  
1: 1  
Player 1:  
1: 2  
Player 2:  
1: 8  
Player 1:  
1:
```

- After every move, the plot of the playing field is updated accordingly. The cells selected by Player 1 are marked with a red circle, the ones selected by Player 2 with a green cross. After the four moves given in the example above, it should look like this:

7	×	9
4	⑤	6
×	②	3

- Every move must be valid, i.e., an integer between 1 and 9, which is not marked yet. Otherwise, the function reacts with the following message and the respective player has to insert a cell, again.

```
Player 1:
1: 0
Cell not valid. Again:
1:
```

- The game ends (i.e., the function terminates) as soon as one player wins or if there is no free cell left on the playing field. The following messages (appropriately adapted, if Player 2 wins) should be printed in the console, respectively:

```
[1] "Player 1 wins!"
```

or

```
[1] "Game ends in a tie!"
```

- In particular, after every move it must be tested, whether one player won.
- Hints:
 - Swap (self-contained) subproblems in separate functions, which are then called in the main function `tic_tac_toe` (e.g. for checking if a player won).
 - Test whether the function fulfills all requirements with different inputs.

Time and date handling & classical time series analysis

Exercise 8 (Date-time objects):

Create date-time objects which hold the correct date and/or time for

05/29/2015

Friday, 29 May 2015

Friday, 29 May 2015 05:50

Friday, 29 May 2015 05:50 AM

Friday, 29 May 2015 05:50:06

05/29/2015 05:50

05/29/2015 5:50 AM

05/29/2015 05:50:06

Fri, 16 May 2015 05:50:06 GMT

Exercise 9 (Trend):

The data set <http://galbithink.org/telcos/telephones-1876-1981.xls> contains the number of telephones in the U.S. at the close of the year from 1876 to 1981. Read the data via:

```
library("rio")
phone <- import("http://galbithink.org/telcos/telephones-1876-1981.xls",
                skip = 2) # skip first two lines
```

Plot the development of the number of telephones (**telephones**) per year (**year**). Estimate a linear and an exponential trend and add both trend lines to the plot. Which type fits better?

Exercise 10 (A function for trend and seasonality):

Write a function

```
plot.cta <- function(data, linear = NA, figure = 0) {
  # Your part
}
```

which estimates a classical time series model. The arguments of the function determine the model specification as follows.

- With `linear = NA` it uses the mean of `data` as trend,
- with `linear = TRUE` it computes a linear trend,
- with `linear = FALSE` it computes an exponential trend,
- with `figure = 0` it computes no seasonality component,

- with `figure > 0` it computes an additive seasonality model and the period length is equal to `figure`, and
- with `figure < 0` it computes a multiplicative seasonality model and the period length is equal to `-figure`.

The function generates a plot of the data (black) and the fitted values resulting from the model (red). Furthermore, it computes R^2 and displays it in the main of the plot.

Exercise 11 (Trend and seasonality):

The following table shows the number of home slaughterings of pigs (in 1,000) of a federal state for the quarters of the years 1990 to 1992. Use your function from Exercise 10 to compare linear and exponential trends with additive and multiplicative seasonality with appropriate period length, respectively. Which of the four models is the best one?

Year	Quarter	Number
1990	1	14
	2	6
	3	4
	4	13
1991	1	12
	2	5
	3	4
	4	12
1992	1	11
	2	5
	3	4
	4	12